

```

# sparse backend
solver = setup_sparse_solver(
    n_var=nz, n_ineq=nineq, n_eq=neq,
    sparsity_patterns={"G":G, "A":A},
    fixed_elements={"P":P, "q":q},
    options=options
)

# then just run!
sol = solver(G=G, h=h, A=A, b=b, warmstart=None)

# options example
options = {
    "diff_mode": "fwd" # or "rev"
    "solver":{
        "backend": "qpsolvers",
        "solver_name": "piqp"
    }
}

```

```

# constant constraint
cst1 = Constraint(
    cst_type = "equality",
    lhs = jnp.array([1,0]),
    rhs = jnp.array(0.1)
)

# parametric constraint
rhs = Variable("rhs", shape=(1,))
cst2 = Constraint(
    cst_type = "inequality",
    lhs = jnp.array([1,0]),
    rhs = lambda v: v["rhs"]
    v_in_rhs = {"rhs": rhs}
)

# slack can be added easily
cst.add_slack(w_quad=10,w_lin=10)

# constraints can be summed together
cst = cst1 + cst2

```

```

# define numerical bounds
x_min = jnp.array([X_MIN] * NX, dtype=jnp.float64)
u_min = jnp.array([U_MIN], dtype=jnp.float64)

# dimensions
n_u, n_x, horizon = 1, 2, 10

# state bounds (with slack)
x_bnds_constraint = box_bounds(
    x_min=x_min, x_max=-x_min,
    n_u=n_u, horizon=horizon
).add_slack(w_quad=10, w_lin=10)

# input bounds
u_bnds_constraint = box_bounds(
    u_min=u_min, u_max=u_max,
    n_x=n_x, horizon=HORIZON_MPC
)

# combine together
bnds_constraint = x_bnds_constraint + u_bnds_constraint

```

```

# define initial condition symbolically
theta_var = Variable("theta", shape=(n_theta,))
x0_var = Variable("x0", shape=(NX,))

# define dynamics (theta-dependent)
def a_(theta):
    return jnp.array([
        [theta[0], theta[1]],
        [theta[2], theta[3]]
    ])
def b_(theta):
    return jnp.array([
        [theta[4]],
        [theta[5]]
    ])

# Parametric LTI dynamics
lhs_dyn = lambda v: build_lti_lhs(
    A=a_(v["theta"]), B=b_(v["theta"]),
    horizon=horizon_mpc
)
rhs_dyn = lambda v: build_lti_rhs(
    A=a_(v["theta"]), x0=v["x0"],
    horizon=horizon_mpc
)
dyn_constraint = Constraint(
    cst_type="equality",
    lhs=lhs_dyn, v_in_lhs={"theta":theta_var},
    rhs=rhs_dyn, v_in_rhs={"theta":theta_var, "x0":x0_var}
)

# Fixed LTI dynamics
A = jnp.array([[1.0, 1.0], [0.0, 1.0]])
B = jnp.array([[0.0], [1.0]])
dyn_constraint = lti_dynamics(A=A, B=B, x0=x0_var)

```

```

# constant cost
mpc_cost = Cost(
    q_mat = jnp.eye(2),
    q_vec = jnp.zeros(2),
    c = jnp.array(0.0)
)

# parametric cost
q_mat_diag = Variable("q_diag", shape=(2,))
x_ref       = Variable("x_ref", shape=(2,))

mpc_cost = Cost(
    q_mat = lambda v: jnp.diag(v["q_diag"]),
    q_vec = lambda v: -2*jnp.diag(v["q_diag"])*v["x_ref"],
    c = jnp.array(0.0),
    v_in_q_mat={"q_diag": q_mat_diag},
    v_in_q_vec={"q_diag": q_mat_diag, "x_ref": x_ref},
    v_in_c=None
)

# state-tracking helper (accepts lists of matrices too)
stage_cost = state_tracking_cost(
    Q=Q, R=R,
    r_x=jnp.zeros(NX), r_u=jnp.zeros(NU),
    x0=x0_var, horizon=HORIZON_MPC
)

```

```

# define plant
plant = Dynamics(
    true_fun=lambda x, u, params: A @ x**2 + B @ u + params["d"],
    nx=2, nu=1,
    true_params_spec=[Variable("d", shape=(2,))]
)

# define linearized dynamical constraints
dyn = nonlinear_dynamics(
    dyn=plant,
    x_nominal=x_nom_var,
    u_nominal=u_nom_var,
    horizon=HORIZON_MPC
)

```

```

# construct MPC
mpc = MPCProblem(
    costs=[cost],
    constraints=[dyn, bnds_state, bnds_input],
    outputs={
        "u0": slice(HORIZON_MPC * NX, HORIZON_MPC * NX + NU),
        "x": slice(0, HORIZON_MPC * NX),
        "u": slice(HORIZON_MPC * NX, HORIZON_MPC * (NX + NU)),
    },
    mode="sparse",
)

# get solver
solver = setup_sparse_solver(
    n_var=mpc.n_dec,
    n_ineq=mpc.n_ineq,
    n_eq=mpc.n_eq,
    sparsity_patterns=mpc.sparsity_pattern,
    fixed_elements=mpc.fixed_elements,
    options={"diff_mode": "rev"},
)
mpc_solver = mpc.with_solver(solver)

```

```

def init(inputs):

    # initial linearization
    x_seed = jnp.tile(inputs["x0"], (HORIZON_MPC, 1))
    u_seed = jnp.zeros((HORIZON_MPC, NU))

    # solve once for better initialization
    sol0 = mpc.solve({
        "p"      : p,
        "x0"     : inputs["x0"],
        "x_nom"  : x_seed,
        "u_nom"  : u_seed,
    })

    # extract solution
    x_nom = sol0["x"]["x"].reshape((HORIZON_MPC, NX))
    u_nom = sol0["x"]["u"].reshape((HORIZON_MPC, NU))

    return {"x": inputs["x0"], "x_nom": x_nom, "u_nom": u_nom, \
+ "prepared": prep}

```



```

def step(carry, k):

    # A. Solve the MPC.
    sol = mpc.solve({
        "p"      : p,
        "x0"     : carry["x0"],
        "x_nom"  : carry["x_seed"],
        "u_nom"  : carry["u_seed"],
    })
    u = sol["x"]["u0"]

    # B. Step the true environment.
    x_next = plant.step(carry["x"], u)

    # C. Build the linearisation trajectory for the NEXT step.
    x_sol = sol["x"]["x"].reshape((HORIZON_MPC, NX))
    u_sol = sol["x"]["u"].reshape((HORIZON_MPC, NU))

    new_carry = {
        "x":      x_next,
        "x_nom":  x_sol,
        "u_nom":  u_sol,
        "prepared": carry["prepared"],
    }
    return new_carry, {"x": carry["x"], "u": u}

```

```

def finalize(final_carry, logs):

    # get state and input trajectories
    xs, us = logs["x"], logs["u"]

    # compute cost
    cost = swingup_task_cost(xs, us)

    return {"cost": cost, "xs": xs, "us": us}

# define closed-loop manually ...
sim = ClosedLoop(
    init=init,
    step=step,
    finalize=finalize
    n_steps=HORIZON_SIM
)

sim_out = sim.run({"p":p_init, "x0": x0_init})

# ... or use the simulator builder
simulate_closed_loop_fn = build_closed_loop_simulator(
    mpc=mpc,
    plant=plant,
    trajectory_cost_fn=swingup_task_cost,
    horizon_sim=HORIZON_SIM,
    horizon_mpc=HORIZON_MPC,
    nx=NX,
    nu=NU,
    options={
        "warmstart_first_mpc": True,
        "linearization_mode": "trajectory_shifted"
    }
)

```